VRS NLA Lookup API

Integration guide — calling from another Replit app

Generated 2026-05-28

Endpoint

Method: `GET`
Path: `/api/external/service-order/{SO}/nla`
Auth: `x-api-key header (machine-to-machine, no JWT)`
SO format: `DDDD-SSSSSSSS  (e.g. 8175-12345678)`

Returns every parts_nla submission for the given service order, newest first, regardless of ticket status.

Step 1 — Add secrets to the OTHER Replit app

In the calling app, open the Secrets pane (lock icon in the sidebar) and add two entries:

Key: `VRS_LOOKUP_API_KEY`
Value: `vrs_sMk6AGRRubsQFRC9lSF-E71ST-4FKCoFkpSsByfwVoU`

Key: `VRS_API_BASE`
Value: `https://<your-vrs-app>.replit.app`

Replace <your-vrs-app> with this app's deployed URL — find it in the Deployments tab after publishing. Do NOT use the dev .repl.co preview URL.

Step 2 — Paste this helper into the other app

File: server/vrsLookup.ts (TypeScript, works on Node 18+)

```
const VRS_BASE = process.env.VRS_API_BASE;
const VRS_KEY = process.env.VRS_LOOKUP_API_KEY;

export type VrsNlaSubmission = {
  id: number;
  serviceOrder: string;
  applianceType: string;
  requestType: string;
  ticketStatus: string;
  technician: { id: number; name: string; racId: string | null; phone: string | null };
  agent: { id: number; name: string } | null;
  escalatedBy: { id: number; name: string } | null;
  nlaResolution: string | null;
  nlaFoundPartNumber: string | null;
  partNumbers: string | null;
  technicianMessage: string | null;
  agentNotes: string | null;
  createdAt: string | null;
  claimedAt: string | null;
  statusChangedAt: string | null;
};

export type VrsNlaLookup = {
  serviceOrder: string;
  followedNlaProcess: boolean;
  nlaSubmissionCount: number;
  submissions: VrsNlaSubmission[];
};

export async function lookupNlaBySO(serviceOrder: string): Promise<VrsNlaLookup> {
  if (!VRS_BASE || !VRS_KEY) {
    throw new Error("VRS_API_BASE and VRS_LOOKUP_API_KEY must be set");
  }
  if (!/^\d{4}-\d{8}$/.test(serviceOrder)) {
    throw new Error(`Bad SO format: ${serviceOrder} (expected DDDD-SSSSSSSS)`);
  }
  const url = `${VRS_BASE}/api/external/service-order/${serviceOrder}/nla`;
```

Step 3 — Call it from anywhere in that app

```
import { lookupNlaBySO } from "./vrsLookup";

const result = await lookupNlaBySO("8175-12345678");

if (result.followedNlaProcess) {
  console.log(`Tech followed NLA process — ${result.nlaSubmissionCount} submission(s)`);
  for (const s of result.submissions) {
    console.log(`  #${s.id}  status=${s.ticketStatus}  tech=${s.technician.name}  ` +
                `resolution=${s.nlaResolution ?? "none"}`);
  }
} else {
  console.log("No NLA submissions found for this SO.");
}
```

Step 4 — Quick shell sanity check

```
curl -H "x-api-key: vrs_sMk6AGRRubsQFRC9lSF-E71ST-4FKCoFkpSsByfwVoU" \
  https://<your-vrs-app>.replit.app/api/external/service-order/8175-12345678/nla
```

Expect a 200 with JSON containing "followedNlaProcess": true (for an SO that has NLA history).

Response shape

```
{
  "serviceOrder": "8175-12345678",
  "followedNlaProcess": true,
  "nlaSubmissionCount": 1,
  "submissions": [{
    "id": 49,
    "ticketStatus": "queued",
    "applianceType": "refrigeration",
    "technician": { "id": 10, "name": "...", "racId": "...", "phone": "..." },
    "agent": { "id": 7, "name": "..." } | null,
    "escalatedBy": { "id": 10, "name": "..." } | null,
    "nlaResolution": "part_found_vrs_ordered",
    "nlaFoundPartNumber": null,
    "partNumbers": "[\"WPW123456789\"]",
    "technicianMessage": "...",
    "agentNotes": "...",
    "createdAt":  "2026-04-05T17:43:00.756Z",
    "claimedAt":  "2026-04-22T21:24:41.742Z",
    "statusChangedAt": "2026-04-27T12:32:54.488Z"
  }]
}
```

followedNlaProcess is the headline boolean — true if any parts_nla submission exists for that SO (any status).

Status codes

- 200 — query succeeded; check nlaSubmissionCount
- 400 — bad SO format
- 401 — missing or invalid x-api-key
- 503 — VRS_LOOKUP_API_KEY is not set on this app
- 500 — DB error

Rotating the key

- Update VRS_LOOKUP_API_KEY in Replit Secrets on this (VRS) app.
- Update the same value in the calling app's secrets.
- No restart needed on either side — the value is read per request.

Common gotchas

- 401 !’ secret values don't match. Check both apps for trailing spaces.
- 503 !’ secret isn't set on the VRS app yet. Set it and redeploy.
- 400 !’ SO must be 4 digits, hyphen, 8 digits (e.g. 8175-12345678).
- Network error !’ ensure VRS_API_BASE points to the published deployment URL, not the dev preview.

Security note

The payload includes technician name / RAC ID / phone and free-text agent notes — wider than the minimum needed for a yes/no check. Treat the key like a password: keep it server-side, rotate when a developer leaves, and never embed it in client-side code. If you ever want a stripped-down summary endpoint, ask and we'll add one.